# Jellyfin on Android / Termux

Complete Debug & Fix Log — proot Ubuntu/Debian

*For future reference · Every attempt, failure, fix, and final working solution*

---

## Goal & Environment

Run Jellyfin media server on Android using Termux + proot Linux distro, without root access.

| | |
|---|---|
| **Device** | Redmi (Android) |
| **Method** | Termux + proot-distro (no root) |
| **Goal** | Self-hosted Jellyfin for movies/media on LAN |
| **Architecture** | aarch64 (ARM64) |

---

## Phase 1 — Termux Native Install

| PHASE 1 Termux-native Jellyfin [FAILED ] | |
|---|---|
| **Attempt** | Install jellyfin-server directly in Termux without proot |
| **Error** | `libhostfxr.so not found You must install .NET runtime` |
| **Root Cause** | Termux uses Android bionic libc. Jellyfin requires glibc + full .NET runtime. .NET is not available or compatible in Termux native environment. |
| **Conclusion** | Termux-native Jellyfin = dead end. proot required. |

---

## Phase 2 — Switch to proot Ubuntu

| PHASE 2 proot Ubuntu (Resolute 26.04) [FAILED ] | |
|---|---|
| **Attempt** | Install proot-distro, then install Ubuntu via proot-distro install ubuntu |
| **Problem** | Got Ubuntu 26.04 'Resolute Raccoon' — a bleeding-edge/unreleased version |

| | |
|---|---|
| **Error** | `jellyfin-ffmpeg7 depends on libbluray2 (not installable) Package 'libbluray2' has no installation candidate` |
| **Root Cause** | Jellyfin repo targets Ubuntu 20.04 / 22.04 (jammy). Ubuntu 26.04 replaced libbluray2 with a newer version. proot-distro defaulted to latest Ubuntu, not a stable LTS. |
| **Conclusion** | Wrong base distro = dependency hell. Need older stable Ubuntu or Debian. |

---

## Phase 3 — Fix Repo + Certificates (Ubuntu)

| **PHASE 3 GPG Key & CA Certificates Fix [PARTIAL ]** | |
|---|---|
| **Attempt** | Manually add Jellyfin repo with GPG key to Ubuntu proot |
| **Error** | `Certificate verification failed No system certificates available` |
| **Root Cause** | Fresh proot container has ca-certificates package but update-ca-certificates never ran. No system certs initialized. |
| **Fix Applied** | `apt install ca-certificates -y update-ca-certificates` |
| **Outcome** | Repo started working. Packages could be fetched. But install still failed due to dependency mismatch (libbluray2). |
| **Key Lesson** | Always run update-ca-certificates in a fresh proot container before trying to hit HTTPS repos. |

---

## Phase 4 — Ubuntu Jammy Repo Attempt

| **PHASE 4 Jellyfin on Ubuntu Jammy repo [FAILED ]** | |
|---|---|
| **Attempt** | `Add jammy repo: https://repo.jellyfin.org/ubuntu jammy main apt install jellyfin -y` |
| **Error** | `Segmentation fault (after successful install)` |
| **Root Cause** | Jellyfin uses .NET runtime + native libs. proot provides partial syscall emulation only. Android kernel != full Linux — memory execution permissions differ. Binary installs fine but crashes at runtime. |
| **Key Lesson** | Install success ≠ runtime compatibility. Always test execution, not just install. |
| **Conclusion** | proot + .NET segfault issue — needs specific env vars to fix. |

---

## Phase 5 — Wrong Download Attempts

| PHASE 5A Downloaded Source Code Instead of Binary [FAILED ] | |
|---|---|
| **Mistake** | Downloaded Jellyfin GitHub source code repo instead of compiled release binary |
| **Symptom** | `./jellyfin → No such file or permission denied` |
| **Root Cause** | GitHub repo = source code. Needs to be compiled. Not the same as a release artifact. |
| **Fix** | Always download from Releases page, not the main repo zip. |

| PHASE 5B Corrupted .deb Download [FAILED ] | |
|---|---|
| **Mistake** | Downloaded a .deb file that was actually an HTML error page |
| **Symptom** | `dpkg: error: 'jellyfin.deb' is not a Debian format archive` |
| **Root Cause** | URL redirected or returned HTML instead of binary. File saved was HTML, not a .deb. |
| **Fix** | `Always verify file with: file jellyfin.deb — should say 'Debian binary package'` |

---

## Phase 6 — Switch to Debian (Trixie 13)

| PHASE 6 proot Debian 13 (Trixie) [FAILED ] | |
|---|---|
| **Attempt** | `proot-distro install debian → got Debian 13 Trixie Added bookworm Jellyfin repo` |
| **Error** | `jellyfin-ffmpeg7 depends on libvpx7 (>= 1.12.0) — not installable jellyfin-server depends on libicu72 — not installable libx265-199 — not installable` |
| **Root Cause** | Jellyfin packages built for Debian 12 (bookworm). Trixie (13) upgraded these libs to newer versions (libvpx9, libicu74 etc.) so the old bookworm deps don't exist. |
| **Fix Attempted** | `echo 'deb http://deb.debian.org/debian bookworm main' >> sources.list apt install libvpx7 libx265-199 libicu72 -y` |
| **Fix Result** | Worked — bookworm libs installed alongside trixie packages |
| **Conclusion** | Mix of bookworm + trixie packages resolved dependency conflict. |

---

## Phase 7 — WORKING SOLUTION

**CONFIRMED WORKING — Debian 13 Trixie + bookworm dependency mix + DOTNET_EnableWriteXorExecute=0**

## Step 1 — Termux Setup (run in Termux, not proot)

```
pkg update -y && pkg upgrade -y
pkg install proot-distro -y
proot-distro install debian
proot-distro login debian
```

## Step 2 — Inside Debian: Update & Install Dependencies

```
apt update && apt upgrade -y
apt install curl gnupg apt-transport-https ca-certificates ffmpeg -y
update-ca-certificates
```

## Step 3 — Add Jellyfin Repo

```
curl -fsSL https://repo.jellyfin.org/debian/jellyfin_team.gpg.key | gpg --dearmor | tee
/usr/share/keyrings/jellyfin-archive-keyring.gpg > /dev/null


echo "deb [signed-by=/usr/share/keyrings/jellyfin-archive-keyring.gpg]
https://repo.jellyfin.org/debian bookworm main" | tee
/etc/apt/sources.list.d/jellyfin.list
```

## Step 4 — Add Bookworm Sources to Fix Dependencies

Critical: Trixie is missing libvpx7, libicu72, libx265-199. Add bookworm as fallback source:

```
echo "deb http://deb.debian.org/debian bookworm main" | tee
/etc/apt/sources.list.d/bookworm.list
apt update
apt install libvpx7 libx265-199 libicu72 -y
```

## Step 5 — Install Jellyfin

```
apt install jellyfin -y
# Downloads ~97MB, installs jellyfin-server + jellyfin-web + jellyfin-ffmpeg7
```

## Step 6 — Create Startup Script (jellyfin.sh)

The critical fix: DOTNET_EnableWriteXorExecute=0 — without this, .NET crashes with segfault on Android kernels that restrict memory execution permissions.

Choose your RAM size:

### 4 GB RAM

```
#!/bin/sh
export DOTNET_EnableWriteXorExecute=0   # CRITICAL — fixes segfault on Android
export DOTNET_GC_SERVER=0
export DOTNET_GC_CONCURRENT=1
export DOTNET_GCHeapHardLimit=800000000
export DOTNET_GCHeapHardLimitPercent=50
jellyfin \
  --webdir /usr/share/jellyfin/web \
  -c "$HOME/.config/jellyfin" \
  -C "$HOME/.cache/jellyfin" \
  -d "$HOME/.local/share/jellyfin" \
  -l "$HOME/.local/share/jellyfin/log" \
  --nonetchange
```

### 6 GB RAM

```
#!/bin/sh
export DOTNET_EnableWriteXorExecute=0
export DOTNET_GC_SERVER=0
export DOTNET_GC_CONCURRENT=1
export DOTNET_GCHeapHardLimit=1200000000
export DOTNET_GCHeapHardLimitPercent=50
jellyfin \
  --webdir /usr/share/jellyfin/web \
  -c "$HOME/.config/jellyfin" \
  -C "$HOME/.cache/jellyfin" \
  -d "$HOME/.local/share/jellyfin" \
  -l "$HOME/.local/share/jellyfin/log" \
  --nonetchange
```

### 8 GB RAM

```
#!/bin/sh
export DOTNET_EnableWriteXorExecute=0
export DOTNET_GC_SERVER=0
export DOTNET_GC_CONCURRENT=1
export DOTNET_GCHeapHardLimit=2000000000
```

```
export DOTNET_GCHeapHardLimitPercent=60
jellyfin \
  --webdir /usr/share/jellyfin/web \
  -c "$HOME/.config/jellyfin" \
  -C "$HOME/.cache/jellyfin" \
  -d "$HOME/.local/share/jellyfin" \
  -l "$HOME/.local/share/jellyfin/log" \
  --nonetchange
```

### Step 7 — Make Script Executable & Run

```
chmod +x jellyfin.sh
./jellyfin.sh
# Wait 30-60 seconds for first launch
```

### Step 8 — Access in Browser

```
# Open Chrome on your phone and go to:
http://localhost:8096
# OR use mDNS:
http://<devicename>:8096

# Complete first-time setup wizard:
# 1. Create admin account
# 2. Add media library (e.g. /sdcard/Movies)
# 3. Done — access dashboard
```

### Every Time You Want to Start Jellyfin

```
# In Termux:
proot-distro login debian

# Inside Debian:
./jellyfin.sh
```

---

## ‘ Key Discoveries & Lessons

| Discovery | Detail |
| --- | --- |

| | |
|---|---|
| **THE fix for segfault** | export DOTNET_EnableWriteXorExecute=0 — Android kernels restrict W^X memory. .NET needs both write and execute on the same memory region. This env var disables that restriction. |
| **--nonetchange flag** | Required for proot — Jellyfin tries to change network config on startup which fails in proot. This flag skips that. |
| **Ubuntu 26.04 = avoid** | proot-distro install ubuntu gives you the latest Ubuntu which is too new. Jellyfin only supports up to 22.04 (jammy). Always pin to a known-good version. |
| **Debian trixie + bookworm mix works** | Trixie is missing old lib versions (libvpx7, libicu72). Adding bookworm as an apt source and installing those specific packages manually resolves it cleanly. |
| **Install success ≠ works** | Jellyfin installed perfectly but segfaulted. Never assume a package install = working runtime. Always test execution. |
| **update-ca-certificates** | Must run this in any fresh proot container before using HTTPS repos. Fresh containers have the package but haven't initialized certs. |
| **Use browser not app** | Jellyfin Android app may struggle to connect to localhost. Use Chrome at http://localhost:8096 for best experience. |
| **Transcoding = CPU only** | No hardware acceleration in proot. For smooth playback use direct play — avoid formats that force transcoding. Browser handles more formats natively than the app. |
| **Keep Termux alive** | Acquire wakelock from Termux notification. Android will kill the proot session if Termux goes to background without it. |
| **Media path in proot** | Your phone storage is accessible at /sdcard/ inside proot. Use /sdcard/Movies etc. as library paths in Jellyfin setup. |
| **Debug with 2>&1 \| head -50** | Running jellyfin 2>&1 \| head -50 captures the actual crash output instead of just showing 'Segmentation fault'. Critical for diagnosing runtime errors. |

---

## ϕ Quick Reference — TL;DR

| What Was Tried | Result | Reason |
|---|---|---|
| Termux native install | **Failed** | .NET / glibc missing |
| Ubuntu 26.04 proot | **Failed** | Too new, libbluray2 gone |
| Ubuntu jammy repo | **Segfault** | Android kernel W^X restriction |
| Source code download | **Failed** | Not a binary, needs compiling |
| Corrupted .deb download | **Failed** | Got HTML file not .deb |
| Debian trixie alone | **Failed** | libvpx7 / libicu72 missing |
| Debian trixie + bookworm libs | **Installed!** | Mixed apt sources resolved deps |

| | | |
|---|---|---|
| ./jellyfin.sh directly | **Segfault** | Missing DOTNET_EnableWriteXorExecute=0 |
| jellyfin.sh + DOTNET fix | **WORKS!** | Correct env vars set, --nonetchange used |

---

*Transcoding is CPU-only in proot (no hardware acceleration). Use direct play where possible.*
*For remote access: use ZeroTier or OpenTunnel. Avoid ngrok (unstable on Termux).*